Assignment 2

Jayesh Narayanan
2023EE11048

Naive-Bayes

Approach:

Created a class with fit and predict methods. In the fit method, the various conditional parameters are trained using a dictionary. Elements are added into the dictionary as and when new words are encountered. The frequency of the words are also noted. The conditional parameters are calculated according to the formula

$$P(w_i \mid C_k) = \frac{N_{w_i|C_k} + \alpha}{\sum_{j=1}^{|V|} N_{w_j|C_k} + \alpha|V|}$$

In the predict method, for each word in the test text, the log probabilities are added to each class using the conditional parameters. Then whichever class has the highest probability is reported as the prediction for that particular entry.

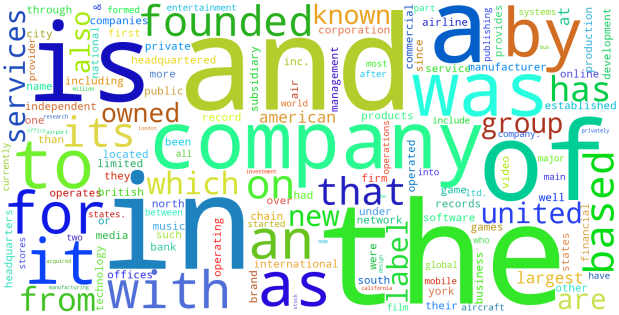
For each subpart, I observed the training and testing accuracy for each subpart of the questions by applying different pre-processing techniques.

Results:

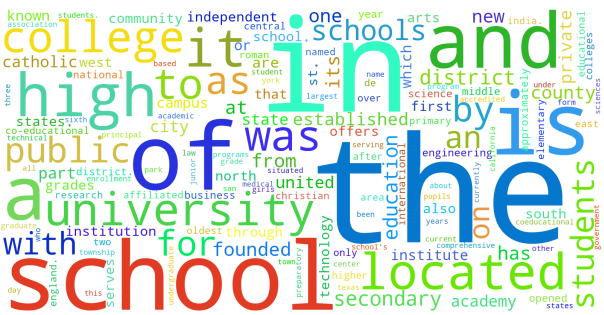
Raw content model:

Training Accuracy: 97.75%, Testing Accuracy: 95.99%

WordClouds:



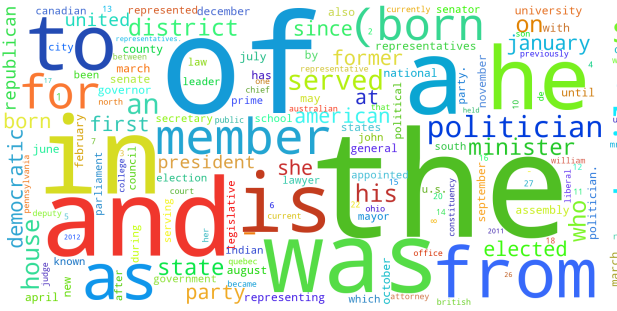
Class 1



Class 2

Class 3

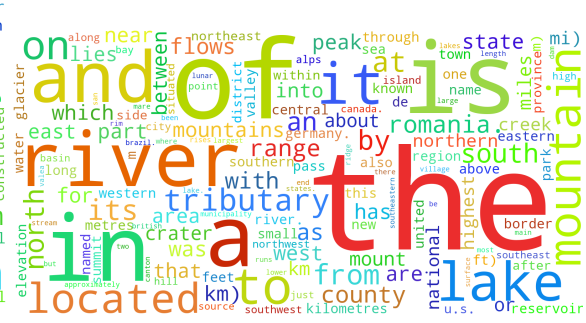
Class 4



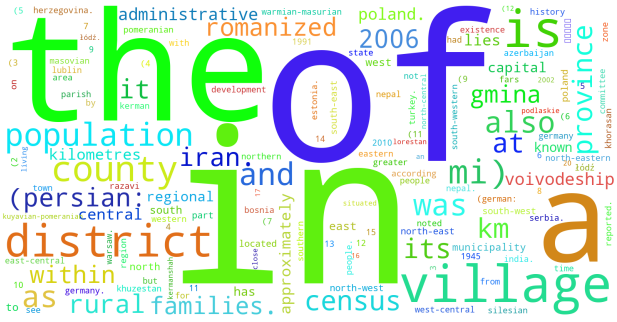
Class 5

Class 6

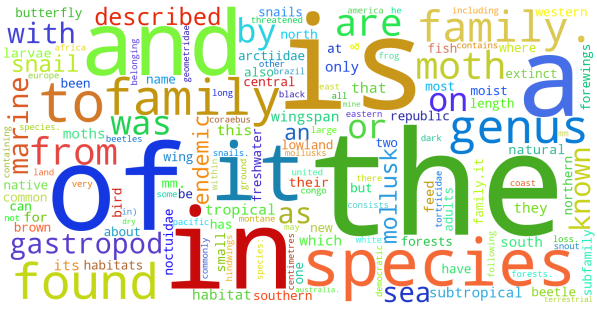
Class 7



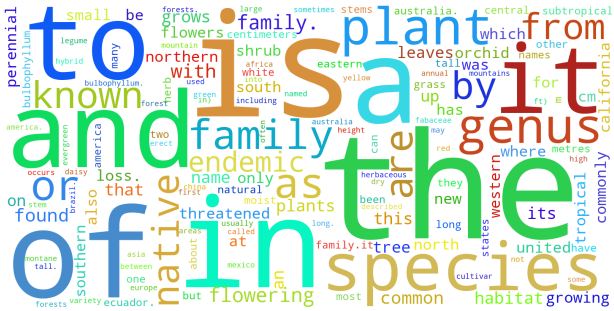
Class 8



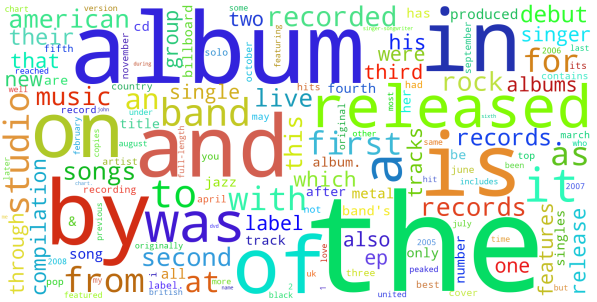
Class 9



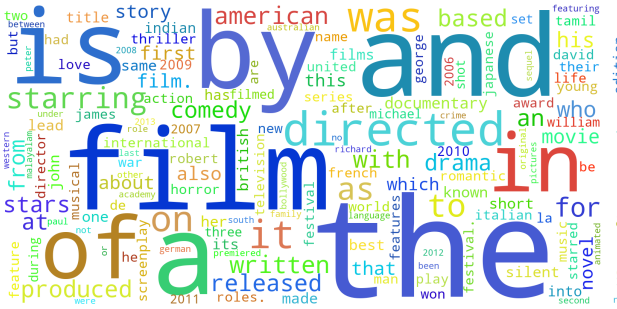
Class 10



Class 11



Class 12



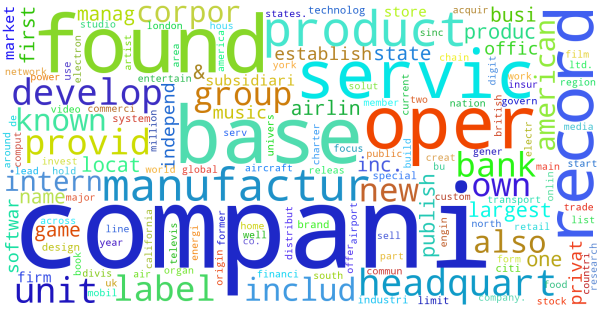
Class 13

Class 14

Processed Content model:

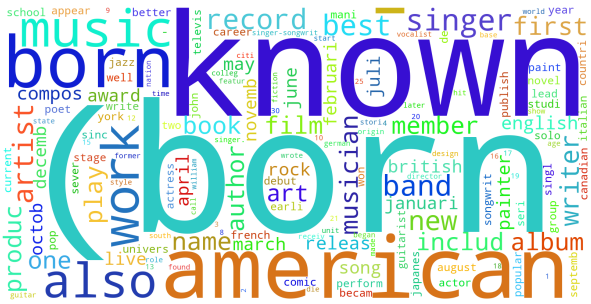
Training Accuracy: 97.41%, Testing Accuracy: 95.31%

Wordclouds:

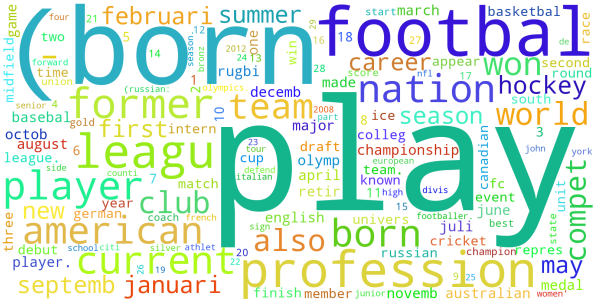


Class 1

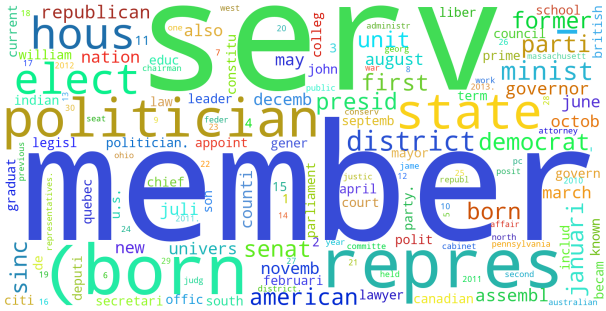
Class 2



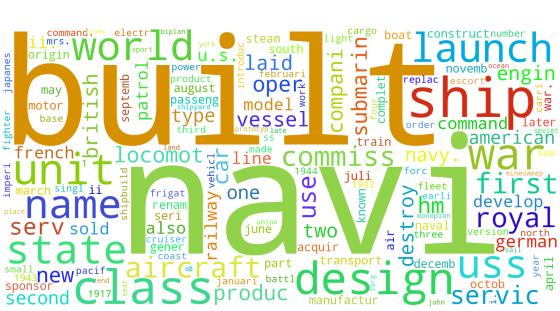
Class 3



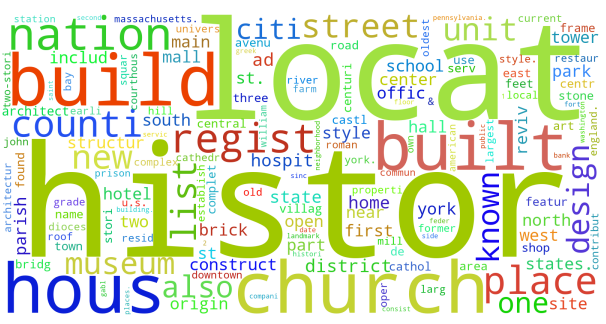
Class 4



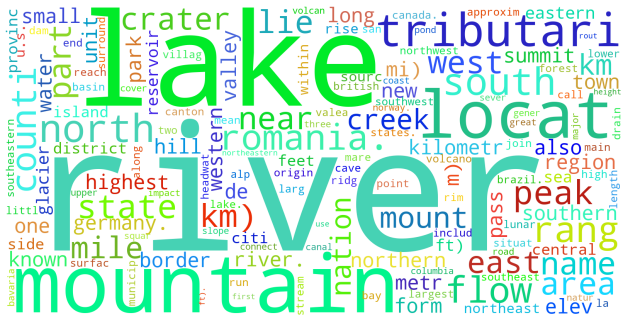
Class 5



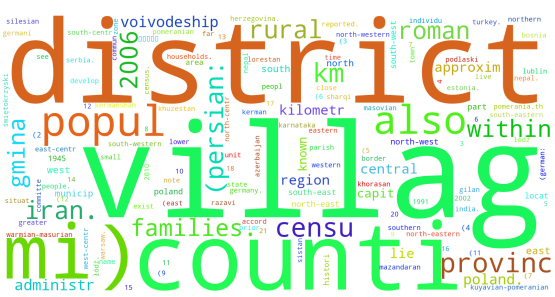
Class 6



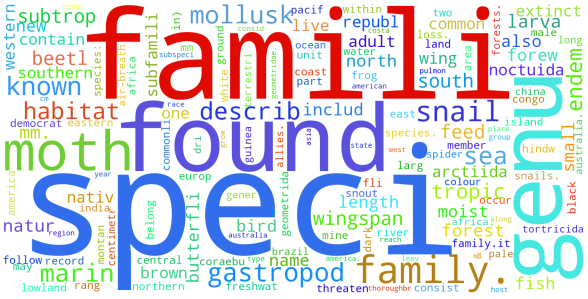
Class 7



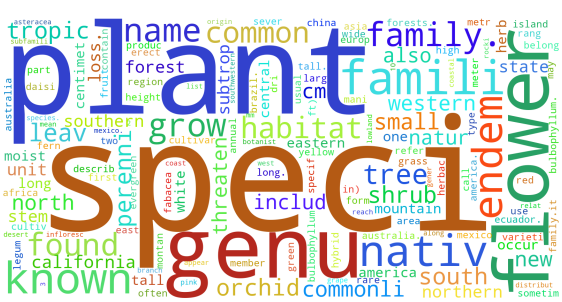
Class 8



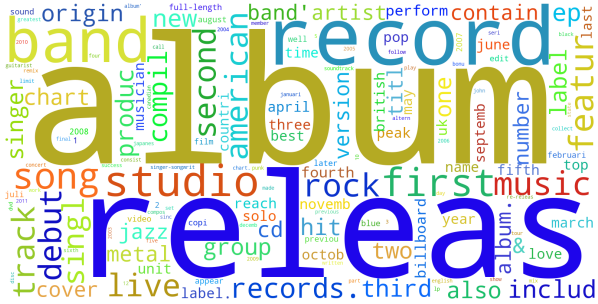
Class 9



Class 10

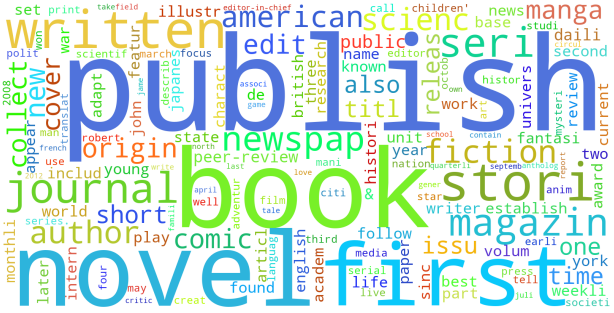


Class 11



Class 12

Class 13



Class 14

Processed Bigram Content Model:

Training Accuracy: 99.38%, Testing Accuracy: 96.14%

Comparison between of Content-based models:

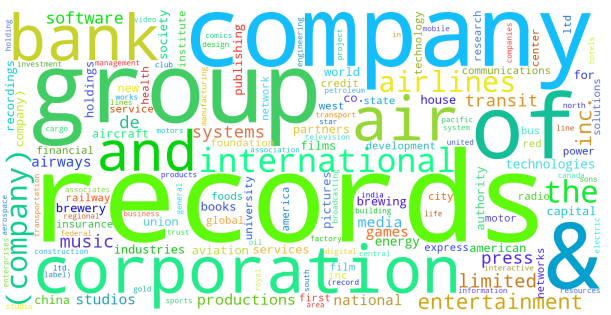
Metric	Raw Content	Processed Content	Processed Bigram + Unigram
Train Accuracy	97.75%	97.41%	**99.38%**
Test Accuracy	95.99%	95.31%	**96.14%**
Precision (macro avg)	0.961	0.956	**0.965**
Recall (macro avg)	0.960	0.953	**0.964**
F1-score (macro avg)	0.960	0.954	**0.965**

We observe that the processed unigram and bigram content model performed the best on all counts

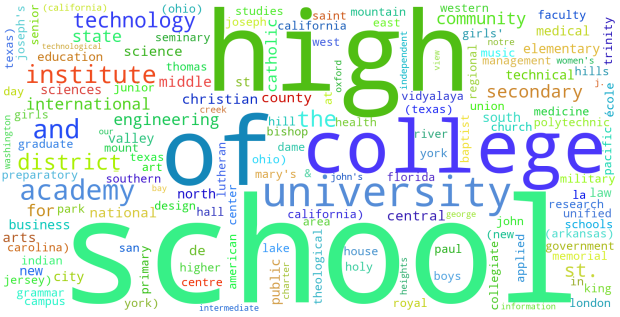
Raw Title model:

Training Accuracy: 90.30%, Testing Accuracy: 65.84%

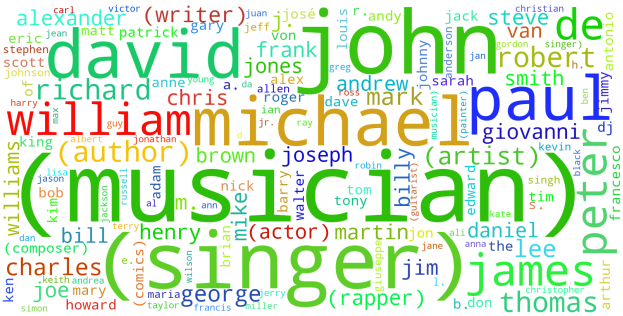
WordClouds:



Class 1

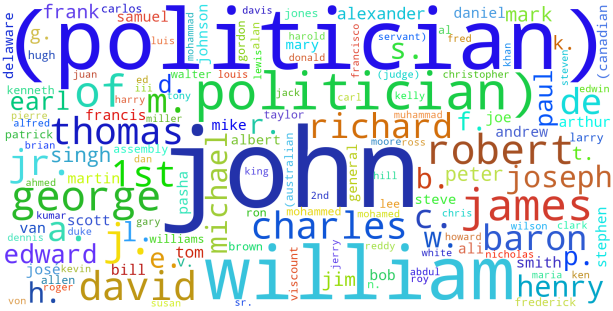


Class 2

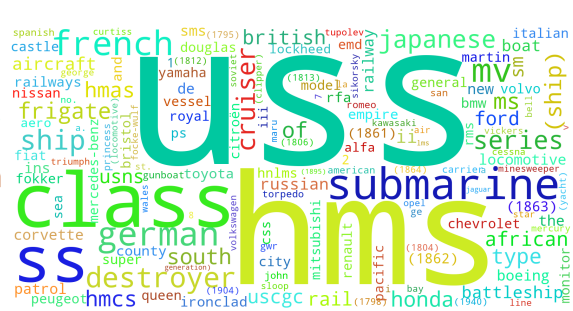


Class 3

Class 4

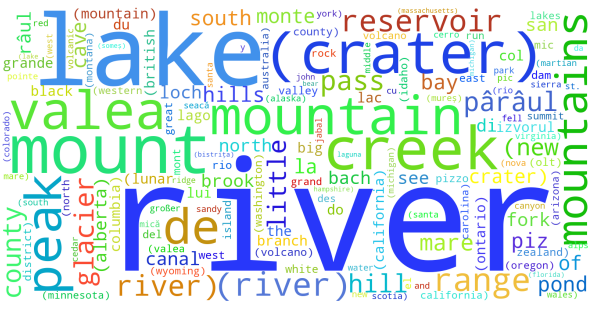


Class 5

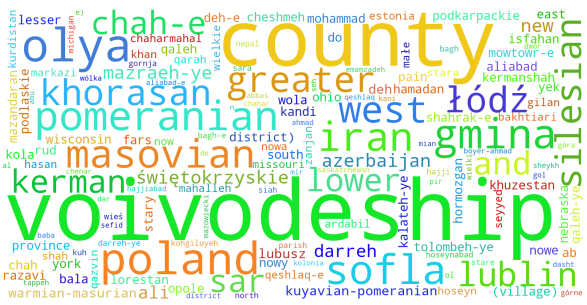


Class 6

Class 7

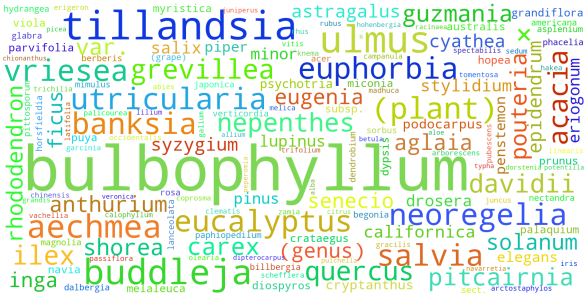


Class 8



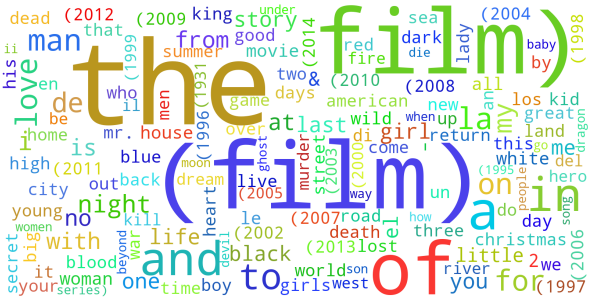
Class 9

Class 10



Class 11

Class 12



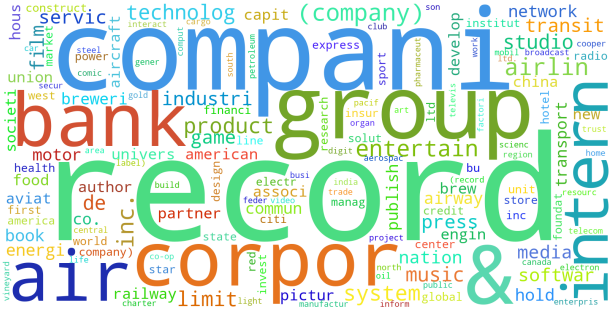
Class 13

Class 14

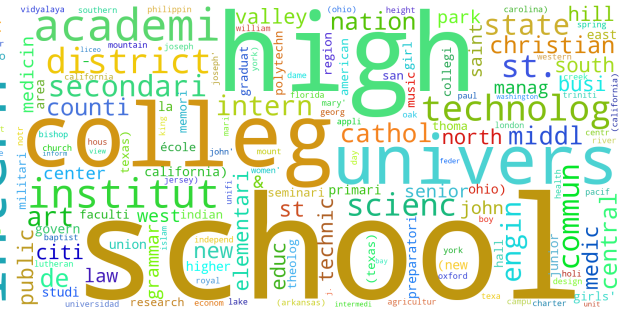
Processed title Model

Training Accuracy: 89.06%, Testing Accuracy: 64.91%

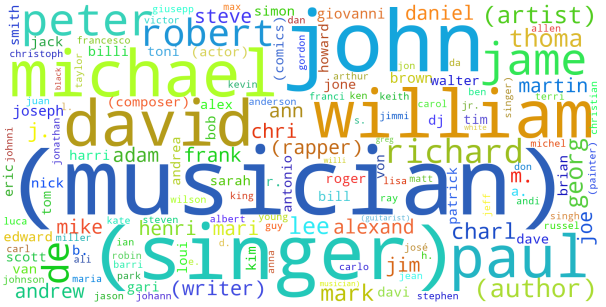
Wordclouds:



Class 1



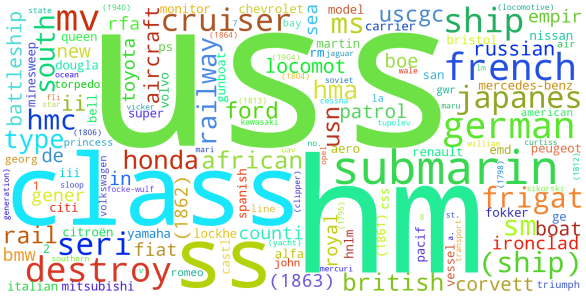
Class 2



Class 3

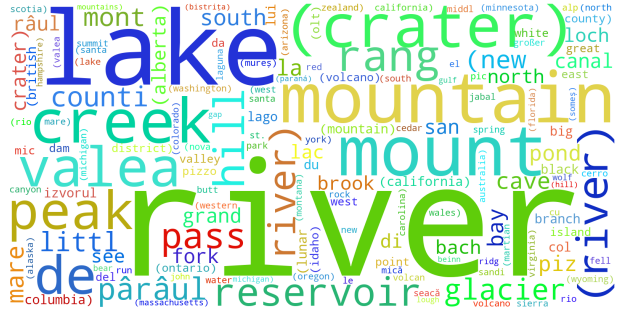
Class 4

Class 5



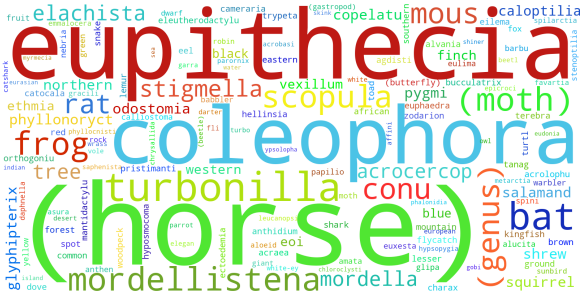
Class 6

Class 7

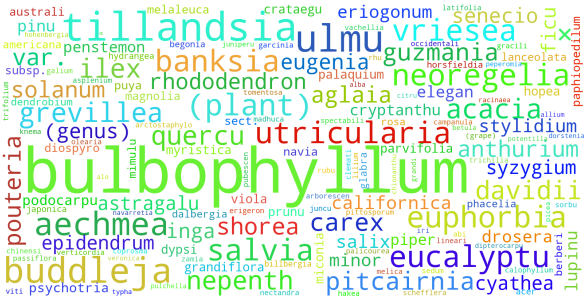


Class 8

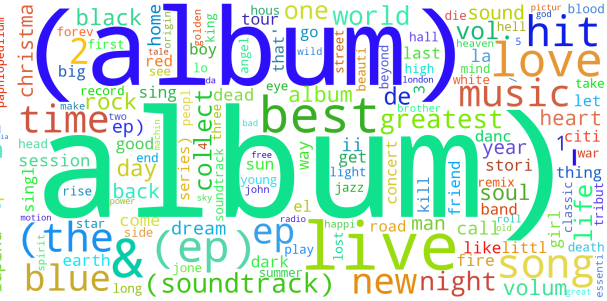
Class 9



Class 10

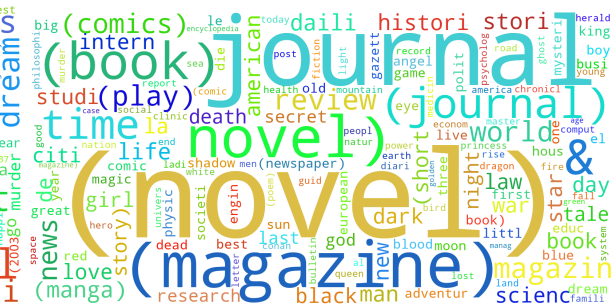


Class 11



Class 12

Class 13



Class 14

Processed Bigram Title Model:

Training Accuracy: 95.36%, Testing Accuracy: 65.03%

Comparison of title-based models:

Metric	Raw Title Data	Processed Title Data	Processed Bigram + Unigram Title Data
Train Accuracy	90.30%	89.06%	95.36%
Test Accuracy	65.84%	64.91%	65.03%
Precision (macro avg)	0.720	0.711	0.712
Recall (macro avg)	0.658	0.649	0.650
F1-score (macro avg)	0.670	0.661	0.662

Analysis:

The Processed Bigram + Unigram Title Data model performs best among the title-based approaches. While the test accuracies of all three models are quite close (~65%), the bigram+unigram model achieves a significantly higher training accuracy (95.36%), reflecting its stronger learning capacity. This improvement arises because bigrams capture contextual relationships between adjacent words — a crucial factor in short text like titles, where word pairs (e.g., *“climate change”*, *“AI research”*) convey specific meanings that single words cannot. However, due to the brevity of titles, the model’s generalization on unseen data remains limited, leading to comparable test performance across models.

Comparison between the best content and best title model:

The best content-based model (Processed Bigram + Unigram) achieves 96.14% test accuracy, while the best title-based model (also Processed Bigram + Unigram) achieves only 65.03%.
Despite both models using the same feature engineering approach, the drastic difference in performance highlights a key issue — overfitting in the title-based model due to the limited information content and short length of titles.

Titles typically contain only a few words, offering much less linguistic and contextual variety than full content. When a complex model like bigram+unigram Naive Bayes is trained on such short text, it can memorize the specific patterns and word pairs in the training data rather than learning generalizable relationships. This leads to high training accuracy (95.36%) but poor generalization on unseen data (65.03%).

In contrast, the content-based model benefits from richer textual information, which allows the bigram and unigram features to generalize better and significantly reduce overfitting. As a result, it achieves both high training accuracy and strong test performance, making it far more reliable and robust than the title-based counterpart.

Joint Model with Shared Parameters

In this setup, we combined the title and content of each document into a single text representation and trained a common model, ensuring that the same set of parameters (θ) was learned for both. This is equivalent to concatenating the two feature sets into a joint feature vector before classification.

Results:

- **Training Accuracy:** 99.49%
- **Testing Accuracy:** 96.23%

Comparison and Observations:
The accuracy obtained using the concatenated model was higher than that of models trained solely on title or content features. The performance improvement demonstrates that concatenating both sources of text provides a richer and more diverse set of features, leading to better generalization.

Compared to title-only models, which tend to overfit due to the limited amount of textual data in titles, the concatenated model benefits from the additional context provided by the content. This helps reduce overfitting and improves the model's ability to distinguish between classes on unseen data. The high training accuracy (≈99.5%) with a slightly lower test accuracy (≈96.2%) still suggests mild overfitting, but substantially less than what is typically observed in title-only models.

Model with Separate Parameters for Title and Content (Maximum Likelihood with Laplace Smoothing)

In this case, we allowed the model to learn **independent parameters** for the title and content — θ(title) and θ(content) — rather than forcing them to share the same parameter space. Using **maximum likelihood estimation** with **Laplace smoothing**, the best-fit expressions for the parameters were computed as:

$$\theta_{y,w}^{(title)} = \frac{N_{y,w}^{(title)} + 1}{\sum_{w'}(N_{y,w'}^{(title)} + 1)}$$

$$\theta_{y,w}^{(content)} = \frac{N_{y,w}^{(content)} + 1}{\sum_{w'}(N_{y,w'}^{(content)} + 1)}$$

Results:

- **Training Accuracy:** 99.51%
- **Testing Accuracy:** 96.52%

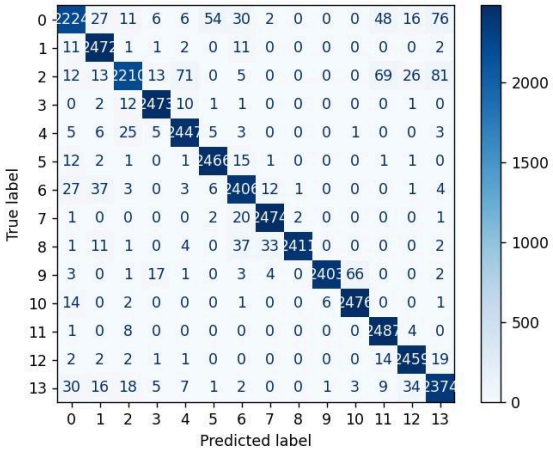
Comparison and Observations:

Allowing the model to learn separate parameters for titles and content slightly improved the test accuracy (from **96.23% to 96.52%**). This improvement indicates that treating title and content as distinct but complementary sources of information is beneficial. Titles often summarize the main topic, while content provides detailed context — learning them separately helps capture both high-level and fine-grained patterns more effectively. In contrast, the concatenation model with shared parameters implicitly assumes that the same word has the same contribution whether it appears in the title or content. This assumption may not always hold — certain words in titles might have more discriminative power than when they appear in the body text. Hence, decoupling the parameters allows the model to adapt more flexibly.

Comparison with baseline model:

Baseline	Accuracy	Notes
Random (uniform)	7.14%	1/14, equally likely classes
Always majority class	7.14%	All classes equally frequent
Your model	96.52%	From validation/test set

Improvement (random)	96.52 - 7.14 = 89.38%	Gain over random baseline
Improvement (majority)	96.52 - 7.14 = 89.38%	Gain over positive baseline



Confusion matrix of best model

Featured Engineered model:

Rationale:

The initial model used unigrams and bigrams as features. To allow the model to capture longer patterns and relationships in the text (e.g., sequences of 3–5 words), we engineered additional features by including trigrams, quadgrams, and pentagrams.

Implementation:

Included 3,4 and 5 grams in the dataset and retrained the best model(seperate params model)

Results:

Training accuracy: 99.97%, Testing accuracy: 96.61%

Analysis:

- The inclusion of higher-order n-grams allowed the model to capture longer-term dependencies in the text, improving its predictive performance.
- The fact that both training and testing accuracy improved suggests the model was previously underfitting slightly on sequences longer than bigrams.
- The model is not overfitting, as test accuracy also increased.
- Feature engineering with longer n-grams is particularly useful for multi-class text classification tasks where context matters.

SVM:

Binary classification:

Approach:

I implemented a **binary Support Vector Machine (SVM) classifier** from scratch using quadratic programming via **CVXOPT**, supporting both **linear and Gaussian (RBF) kernels**. During training, I solve the dual SVM optimization problem to obtain the Lagrange multipliers α\alphaα by maximizing:

$$\max_{\alpha} \; W(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{N} \alpha_i y_i = 0$$

where K(xi,xj) is the kernel function. For the **linear kernel**, K(xi,xj)=xi⊤xj, and the weight vector is computed as:

$$w = \sum_{i=1}^{N} \alpha_i y_i x_i$$

For the **Gaussian (RBF) kernel,** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$

and predictions rely on kernel evaluations between support vectors and new samples. The bias term is calculated as:

$$b = \frac{1}{|\mathrm{SV}|} \sum_{i \in \mathrm{SV}} \left(y_i - \sum_{j \in \mathrm{SV}} \alpha_j y_j K(x_j, x_i) \right)$$

Finally, new samples are classified according to:

$$\hat{y} = \mathrm{sign}\left(\sum_{i \in \mathrm{SV}} \alpha_i y_i K(x_i, x) + b \right)$$

Results:

Linear CVXOPT SVM:

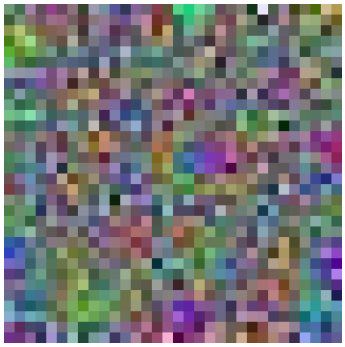
Train Accuracy: 0.9358, **Test Accuracy**: 0.7898

Support Vectors: 1286 (38.57%)

Bias Term: 1.9977, **Training time**: 54.44 sec

Support Vector Images:

Weight vector image:



Gaussian CVXOPT SVM:

Train Accuracy: 0.8596 **Test Accuracy**: 0.8574

Support Vectors: 1811 (54.32%)

Linear and RBF overlap: 1058, **Training time:** 120.36 sec

Support Vector Images:

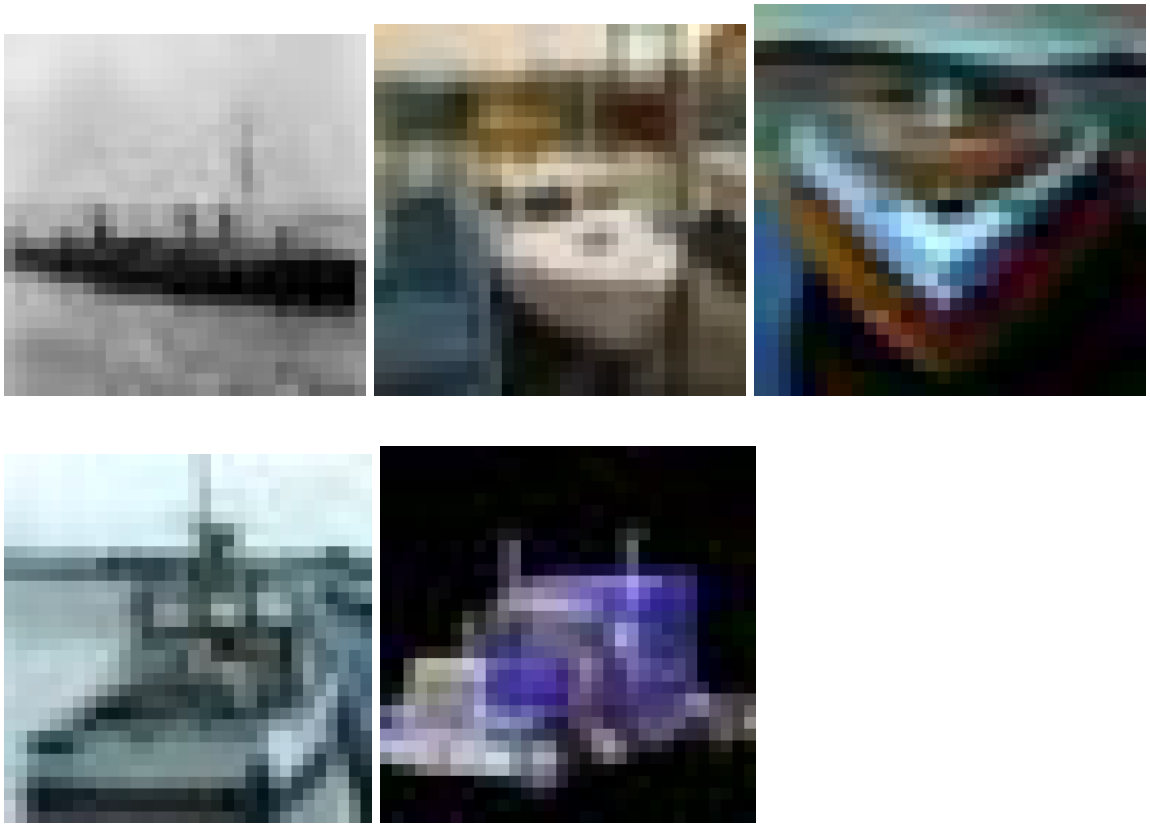
Linear Libsvm:

Train Accuracy: 0.9355, **Test Accuracy**: 0.7913,

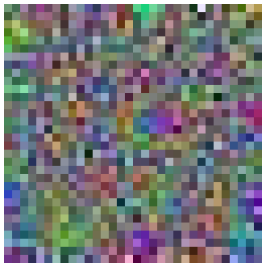
Support Vectors: 1281 (38.42%), **Linear ↔ LIBSVM overlap**: 1281

Bias term: 2.05488065, **Training time**: 12.35 sec

Support vector images:



Weight vector image:



Gaussian Libsvm:

Train Accuracy: 0.9451, **Test Accuracy**: 0.8799

Support Vectors: 1740 (52.18%), **Training time:** 8.19 sec

RBF ↔ LIBSVM overlap: 1570

Support Vector Images:

Analysis:

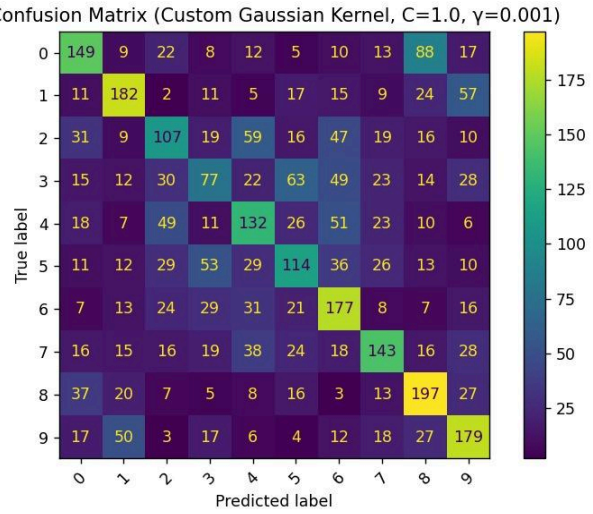
- We observe that the test accuracy of the gaussian kernel is better than that of the linear kernel. This is likely due to the fact that the relationships are captured better with a more complex(higher dimensional kernel) better.
- In both cases, number of support vectors is more in the cvxopt based implementation as compared to the Libsvm implementation
- The weight vector obtained in both cases is the same due to the fact that the cosine similarity is 1
- The bias is also similar
- The computational training time is much lesser in the case of the libsvm based implementation

Multi-class svm:

One vs One classifier:

Test accuracy: 43.75%, **Training time:** 13774.16 seconds

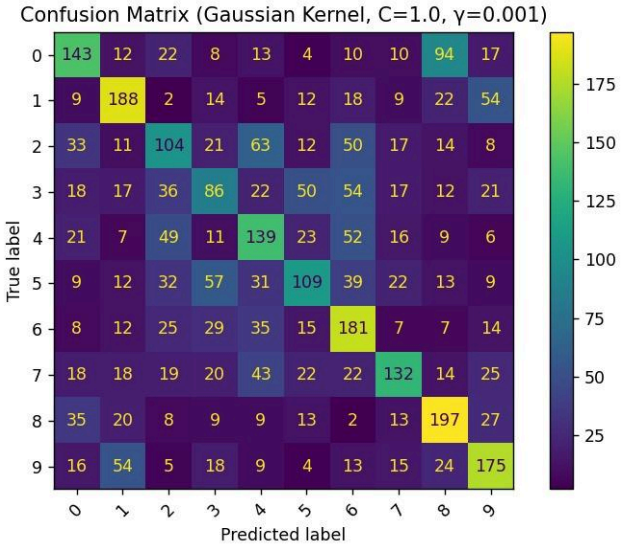
Confusion matrix:



Libsvm classifier:

Test accuracy: 43.66%, **Training time:** 358.71 seconds

Confusion matrix:



Analysis:

From the confusion matrix, we observe that the most error is observed when classifying birds(class 2) and cats(class 3). This is likely due to the fact that there is a lot of variance among birds and cats as compared to other classes like ships, leading to our model underfitting in these classes.

Tuning Parameters:

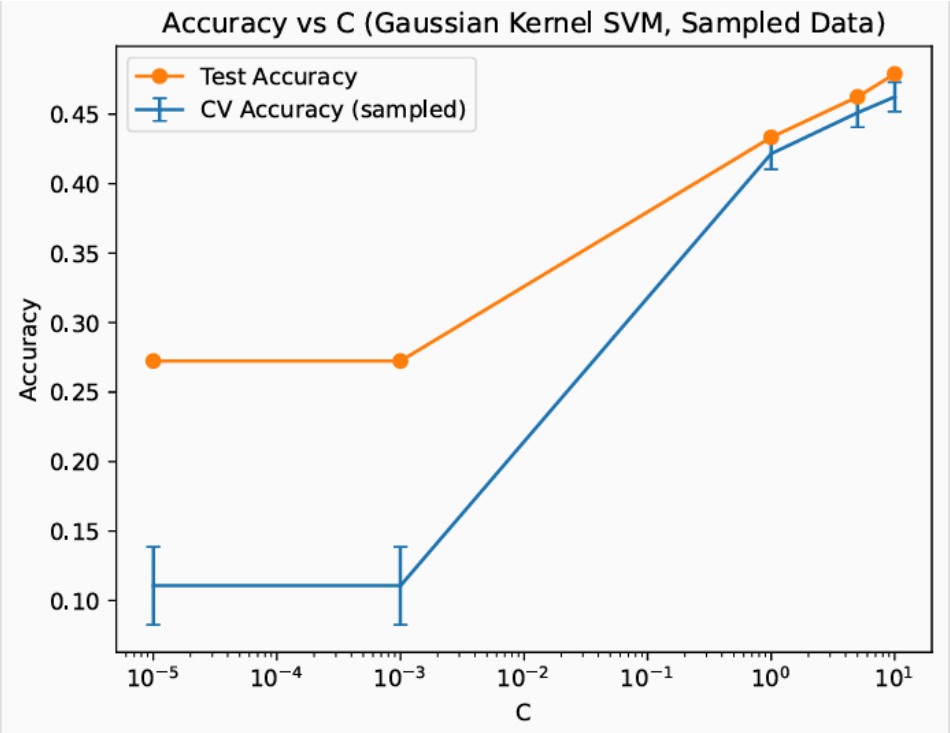
(a) Cross-Validation and Test Performance

To tune the regularization parameter CCC for the Gaussian kernel SVM, I fixed γ=0.001 and varied C over the set {10^−5,10^−3,1,5,10}. For each configuration, I performed 5-fold cross-validation and recorded both the mean validation accuracy and the test accuracy. The results are:

C	CV Mean Accuracy	CV Std	Test Accuracy
1e-05	0.1107	0.0281	0.2724
1e-03	0.1107	0.0281	0.2724
1	0.4215	0.0113	0.4333
5	0.4510	0.0104	0.4625
10	**0.4624**	0.0106	**0.4790**

Visulatization:

The following plot shows both the **5-fold cross-validation accuracy** and **test accuracy** as functions of CCC (x-axis in log scale):



Analysis:

From the graph and the table above, I observed that increasing CCC steadily improved both the cross-validation and test accuracies up to C=10. For very small CCC values (10^−5, 10^−3), the model performed poorly, suggesting strong underfitting due to excessive regularization. As CCC increased, the SVM became more flexible, better fitting the data and reducing bias.

The best 5-fold CV accuracy was achieved at C=10 (0.4624), which also yielded the highest test accuracy of 0.4790. Training the final SVM model using this value of C on the entire training set further confirmed the improvement over previous models. This indicates that moderate regularization (C = 10) provides the best generalization balance for the Gaussian kernel at γ=0.001.